Principles of Machine Learning

Mariette Schönfeld (session 1,3,5) *mariette.schoenfeld@kuleuven.be*
Andrea Mirenda session (2,4) *andrea.mirenda@kuleuven.be*
Isidoro Tamassia (session 6,7) *isidoro.tamassia@kuleuven.be*

Fall, 2026

Exercise Session 4: Optimisation and Support Vector Machines

4.1 Huber Loss Gradient

Huber loss is defined as

$$\mathcal{L}_{H,\delta}(y,\hat{y}) = \begin{cases} \frac{1}{2}(y-\hat{y})^2 & \text{for } |y-\hat{y}| \leq \delta, \\ \delta|y-\hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise.} \end{cases}$$

a. Draw this curve.

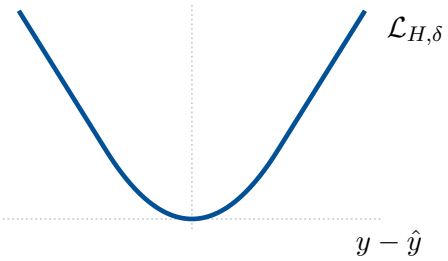
b. Huber is made up of two other loss functions we've seen in the course. Why would you use Huber instead the other losses?

c. Compute the gradient of the Huber loss. Is this function C^1 continuous? Is it C^2 continuous? Do the answers to the previous two questions limit the possible optimization methods?

d. Huber loss has a parameter δ. Which value would you choose for this parameter?

Solution 4.1

a. Parabola in the middle, straight lines with slopes δ as tails.



b. Huber loss has the same robustness characteristics as L1 loss, yet it has the continuous behavior of L2 near the origin.

c. Compute the derivative towards $\hat{y}$ of $\mathcal{L}_{H,\delta}$ We do this by computing the derivative for both parts, using the chain rule, taking $r = (y-\hat{y})$:

$$\frac{\partial}{\partial \hat{y}} \frac{1}{2}(y-\hat{y})^2 = \frac{1}{2}\frac{\partial r^2}{\partial r} \cdot \frac{\partial r}{\hat{y}} = r \cdot -1 = -(y-\hat{y})$$

$$\frac{\partial}{\partial \hat{y}}\left[\delta|y-\hat{y}| - \frac{1}{2}\delta^2\right] = \delta\frac{|r|}{\partial r}\cdot\frac{\partial r}{\partial \hat{y}} = \delta\begin{cases}-1\cdot -1 & \text{for } r<0,\\ \textit{undef.} & \text{for } r=0,\\ 1\cdot -1 & \text{for } r>0\end{cases}$$
$$= \begin{cases}\delta & \text{for } (y-\hat{y})\leq 0,\\ \textit{undef.} & \text{for } (y-\hat{y})=0,\\ -\delta & \text{for } (y-\hat{y})>0.\end{cases}$$

Then, combining the results:

$$\frac{\partial \mathcal{L}_{H,\delta}}{\partial \hat{y}} = \begin{cases}+\delta & \text{for } (y-\hat{y})<-\delta,\\ -(y-\hat{y}) & \text{for } |y-\hat{y}|\leq\delta,\\ -\delta & \text{for } (y-\hat{y})>\delta.\end{cases}$$



Note that the above picture is mirrored around its y-axis because we plot $(y-\hat{y})$, and not $\hat{y}$ on the x-axis. That is, an increase in $\hat{y}$ is a decrease in $(y-\hat{y})$, and thus a movement from right to left in the plot.

Another way to reason about this: gradient descent repeatedly steps in the direction of the negative gradient to minimize a function. If the residual $(y-\hat{y})$ is negative, then we want to increase the residual to improve the prediction, which requires a decrease of $\hat{y}$ (y is fixed).

This function is clearly C^1 continuous, but it is not C^2 continuous:

$$\frac{\partial^2 \mathcal{L}_{H,\delta}}{\partial \hat{y}^2} = \begin{cases}0 & \text{for } (y-\hat{y})<-\delta,\\ 1 & \text{for } |y-\hat{y}|\leq\delta,\\ 0 & \text{for } (y-\hat{y})>\delta.\end{cases}$$

A second-order optimization method like Newton-Raphson would suffer from these non-continuities.

d. The δ parameter clearly has to change according to the domain of y and $\hat{y}$. Friedman[1] suggests using the estimated α quantiles of the distribution of $|y-\hat{y}|$. This gives us a new parameter α that can be chosen fairly arbitrarily without having too much effect (any reasonable quantile is a pretty good estimation of the *scale* of the target variable).

◇

4.2 Squared Loss with Squared Regularization

a. Write down the loss function $\mathcal{L}(y, \boldsymbol{X}w)$ for a linear regression problem with L2 regularization with input matrix $\boldsymbol{X}$ containing the instances in its rows, a column vector with weights w, and a column vector y with target values.

b. Compute the derivative of this loss function.[2]

[1] Friedman, Jerome H. "Greedy function approximation: a gradient boosting machine." Annals of statistics (2001): 1189-1232.

[2] The *Dive Into Deep Learning* book has a great chapter on multivariate calculus.

Solution 4.2

a. The loss is defined as:

$$\begin{aligned}\mathcal{L}(y, \boldsymbol{X}w) &= \|y - \boldsymbol{X}w\|_2^2 + \frac{\lambda}{2}\|w\|_2^2 \\ &= \sum_i (y_i - \boldsymbol{X}_{i*}w)^2 + \frac{\lambda}{2}\sum_j w_j^2,\end{aligned}$$

where $\boldsymbol{X}_{i*}$ is the ith row of $\boldsymbol{X}$, that is, the attribute values of instance i.

b. The derivative is given by:

$$\frac{\partial \mathcal{L}(y, \boldsymbol{X}w)}{\partial w} = \sum_i \begin{bmatrix} -2(y_i - \boldsymbol{X}_{i*}w)X_{i1} + \lambda w_1 \\ \vdots \\ -2(y_i - \boldsymbol{X}_{i*}w)X_{iK} + \lambda w_K \end{bmatrix} = -2\boldsymbol{X}^T(y - \boldsymbol{X}w) + \lambda w$$

◇

4.3 Non-Differentiable Functions: Subgradients

The following definition for convexity was given in the lecture: for any $x, y \in \mathrm{dom}(f)$ and $\alpha \in [0, 1]$:

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y).$$

If a function f is differentiable, f is convex if: [3]

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) \quad \text{for all } x, y \in \mathrm{dom}(f).$$

a. Visually show what these definitions mean for $f(x) = x^2$. Explain how what you have drawn relates to the definition.

b. A **subgradient** of a convex function (also non-differentiable) at a value x is any vector $g \in \mathbb{R}^n$ such that:

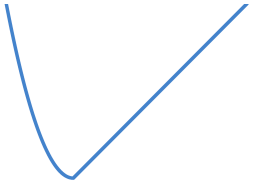
$$f(y) \geq f(x) + g^T(y - x) \quad \text{for all } y \in \mathrm{dom}(f).$$

The **subgradient set** or **subdifferential** is defined as all vectors g for which the above holds:

$$\partial f(x) = \{g \mid g \text{ is a subgradient of } f \text{ at } x\}.$$

Find the subgradient set $\partial f(x)$ for all $x \in \mathbb{R}$ of the function:

$$f(x) = \begin{cases} x^2, & x \leq 0, \\ x, & x > 0. \end{cases}$$

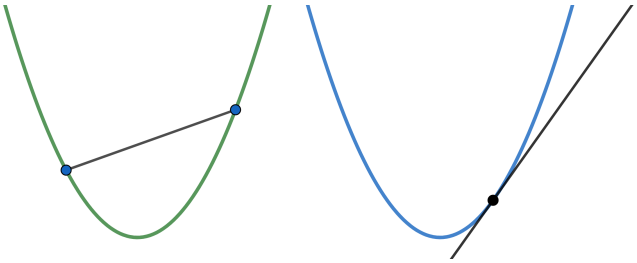


A convex function is minimized at a value x^* if $0 \in \partial f(x^*)$. Can you verify this in the above example?

[3] A great resource for convex optimization is: `http://www.stat.cmu.edu/~ryantibs/convexopt`.

Solution 4.3

a. On the left, the definition from the lectures is illustrated: y-values of the line between two points on the function curve are larger than the function values between those two points. On the right, the y-values of the tangent line in any point of the function curve are less than or equal to the function values.



b. Subgradients:

$$\partial f(x) = \begin{cases} \{2x\}, & x < 0, \\ [0,1], & x = 0, \\ \{1\}, & x > 0. \end{cases}$$

Note that a function f is differentiable at x with gradient value $\nabla f(x)$ if and only if $\partial f(x) = \{\nabla f(x)\}$.

Clearly, the function is minimized in $x = 0$, and $0 \in \partial f(0)$.

◇

4.4 Proximal Methods: Finding Weights for LASSO

Proximal gradient descent is used to optimize convex functions of the form $f(x) = g(x) + h(x)$, where f is non-differentiable, but it can be decomposed into a convex differentiable g and a convex non-differentiable h. We cannot apply the gradient update step from the previous exercise to $f(x)$ in its entirety because it is non-differentiable. However, we can use it for $g(x)$. Consider the following update step, where $g(x)$ is replaced by its quadratic approximation $\tilde{g}(x)$ and $h(x)$ is left unchanged:

$$\begin{aligned} x^{(k+1)} &= \arg\min_z \tilde{g}(z) + h(z) \\ &= \arg\min_z g(x^{(k)}) + \nabla g(x^{(k)})^T (z - x^{(k)}) + \frac{1}{2\rho}\|z - x^{(k)}\|_2^2 + h(z) \\ &= \arg\min_z \frac{1}{2\rho} \underbrace{\|z - (x^{(k)} - \rho\,\nabla g(x^{(k)}))\|_2^2}_{(1)} + \underbrace{h(z)}_{(2)} \end{aligned} \tag{1}$$

We can interpret (1) as: *choose an update z as close to the gradient update $x - \rho\,\nabla g(x)$*, and (2) as: *but also make $h(z)$ small.* Proximal gradient descent uses a proximal mapping to optimize functions of this form. We won't be looking at the general formulation, rather, we will look at an example.

Proximal gradient descent can be used to optimize the LASSO criterion for linear regression. Linear regression with L1 regularization (LASSO) is defined as:

$$\begin{aligned} &\frac{1}{2}\sum_i (y_i - \boldsymbol{X}_{i*}w)^2 + \lambda\|w\|_1 \\ &= \frac{1}{2}\|y - \boldsymbol{X}w\|_2^2 + \lambda\|w\|_1. \end{aligned} \tag{2}$$

The regularization term is not differentiable at zero, so standard gradient descent models aren't applicable. Luckily, the problem can be decomposed into $f = g + h$, where g is the convex differentiable part $\sum_i (y_i - \boldsymbol{X}_{i*}w)^2$, and h is the convex, non-differentiable part $\lambda\|w\|_1$.

We will use the proximal mapping of the L1 norm. But what exactly is a proximal mapping? The general definition of a *proximal mapping* or *proximal operator* of a convex function ϕ is:

$$\text{prox}_\phi(v) = \arg\min_z \left[\phi(z) + \frac{1}{2}\|z - v\|_2^2 \right]$$

Complete books have been written on the subject of proximal methods, so the full details are beyond the scope of this course. What you need to know is that $\text{prox}_\phi(v)$ is well-defined: it has a unique solution because $\|\cdot\|_2$ is strictly convex and we required ϕ to be convex. The sum is therefore also strictly convex, so a unique solution exists. Intuitively, you can view the proximal mapping as a projection to a point in the neighborhood of v that is (1) close to v and (2) minimizes ϕ.

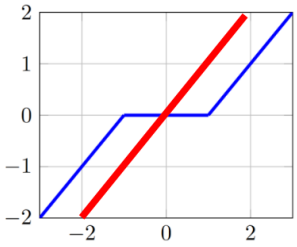
Replacing ϕ in the general definition of a proximal mapping with the L1 norm gives (with weight $\lambda > 0$):

$$\text{prox}_{\lambda\|\cdot\|_1}(v) = \arg\min_z \left[\lambda\|z\|_1 + \frac{1}{2}\|z - v\|_2^2 \right] \tag{3}$$

The above is minimized for:

$$z^* = S_\lambda(v)$$
$$[S_\lambda(v)]_i = \begin{cases} v_i - \lambda & \text{if } v_i > \lambda \\ 0 & \text{if } -\lambda \leq v_i \leq \lambda \\ v_i + \lambda & \text{if } v_i < -\lambda \end{cases} \tag{4}$$

Remember that a convex function ρ is minimized for an input z^* when $0 \in \partial\rho(z^*)$. The S_λ function is called the **soft-thresholding operator**:

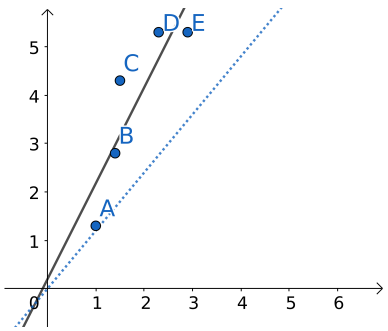


By rewriting the update step in Equation 1 for LASSO (Equation 2), this is equivalent to:

$$\begin{aligned} w^{(k+1)} &= S_{\lambda\rho}\left(w^{(k)} - \rho\nabla g(w^{(k)})\right) \\ &= S_{\lambda\rho}\left(w^{(k)} + \rho\boldsymbol{X}^T(y - \boldsymbol{X}w^{(k)})\right), \end{aligned}$$

where $\nabla g(w) = -\boldsymbol{X}^T(y - \boldsymbol{X}w)$. This is the updating step of the **iterative soft-thresholding algorithm (ISTA)**.

Given the dataset below, $\lambda = 0.2$, learning rate $\rho = 0.01$, and $w^{(0)} = [0.0; 0.5]$, use Python to compute an update step using iterative soft thresholding. Compute the objective function before and after applying the soft-thresholding operator. Is the objective function minimized?



	x	y
A	1.0	1.3
B	1.4	2.8
C	1.5	4.3
D	2.3	5.3
E	2.9	5.3

Solution 4.4 See 'ISTA.py' applying a step of the Iterative Soft Thresholding Algorithm to the weights. Note that, one should not take the soft threshold on w_0! We are interested in promoting sparsity among features, not in the constant term. Regularizing w_0 could negatively affect the model's baseline prediction, leading to biased results.

$$\boldsymbol{X} = \begin{bmatrix} 1.0 & 1.0 \\ 1.0 & 1.4 \\ 1.0 & 1.5 \\ 1.0 & 2.3 \\ 1.0 & 2.9 \end{bmatrix}, \quad y = \begin{bmatrix} 1.3 \\ 2.8 \\ 4.3 \\ 5.3 \\ 5.3 \end{bmatrix}, \quad w^{(0)} = \begin{bmatrix} 0.0 \\ 0.5 \end{bmatrix},$$

$$w^{(1)} = S_{\lambda\rho}\left(w^{(0)} + \begin{bmatrix} 0.1445 \\ 0.2977 \end{bmatrix}\right) = S_{\lambda\rho}\left(\begin{bmatrix} 0.1445 \\ 0.7977 \end{bmatrix}\right) = \begin{bmatrix} 0.1445 \\ 0.7957 \end{bmatrix}.$$

We evaluate the objective function $\frac{1}{2}\|y - \boldsymbol{X}w\|_2^2 + \lambda\|w\|_1$ in $\begin{bmatrix} 0.0 \\ 0.5 \end{bmatrix}$, $\begin{bmatrix} 0.1445 \\ 0.7977 \end{bmatrix}$ and $\begin{bmatrix} 0.1445 \\ 0.7957 \end{bmatrix}$ which gives 24.95, 15.33 and 15.38. That is, the ISTA does slightly worse in the first iteration. Notice how the weights are smaller in magnitude, as expected.

◇

4.5 Iterative Soft Thresholding Algorithm

Suppose you are training a linear regression model with L1 regularization using ISTA with $\eta = 1$ and $\lambda = 1$.

Perform an ISTA update manually with $\eta = 1$ and $\lambda = 1$ using the model and two examples shown in Table 1.

Model: $y = 0.5 + 1.5x_1 + 1.0x_2$

	x_1	x_2	y
Example 1	-1	1	1
Example 2	0.5	1.25	3.0

Table 1: Training data.

Solution 4.5

$$\begin{aligned} \hat{y_1} &= 0.5 + 1.5 \times -1 + 1.0 \times 1 = 0 \\ \hat{y_2} &= 0.5 + 1.5 \times 0.5 + 1.0 \times 1.25 = 2.5 \end{aligned}$$

Note that there is no regularization on w_0. So, you should not apply the soft thresholding operator!

$$\begin{aligned} w_0^{t+1} &= w_0 + \eta(1 \times (y_1 - \hat{y_1}) + 1 \times (y_2 - \hat{y_2})) \\ w_0^{t+1} &= 0.5 + 1 \times (1 \times 1 + 1 \times 0.5) \\ w_0^{t+1} &= 0.5 + 1.5 \\ w_0^{t+1} &= 2.0 \end{aligned}$$

$$
\begin{aligned}
w_1^{t+1} &= s_{\lambda\rho}(w_1 + \eta((x_{1,1}(y_1 - \hat{y_1}) + (x_{2,1}(y_2 - \hat{y_2})) \\
&= s_1(1.5 + 1 \times ((-1)(1) + (0.5)(0.5))) \\
&= s_1(1.5 + 1 \times (-1 + 0.25)) \\
&= s_1(0.75) \\
&= 0
\end{aligned}
$$

$$
\begin{aligned}
w_2^{t+1} &= s_{\lambda\rho}(w_2 + \eta((x_{1,2}(y_1 - \hat{y_1}) + (x_{2,2}(y_2 - \hat{y_2})) \\
&= s_1(1.0 + 1 \times ((1)(1) + (1.25)(0.5))) \\
&= s_1(1.0 + 1 \times (1 + 0.625)) \\
&= s_1(2.625) \\
&= 1.625
\end{aligned}
$$

◇

4.6 A very basic SVM

Consider a two-dimensional classification problem (call the input dimensions a_1 and a_2 and the label y), and consider a dataset with only two instances: x_1 =(1,0,negative) and x_2 =(1,2,positive).

1. Plot the points in a 2-dimensional diagram with a_1 the horizontal axis and a_2 the vertical axis. In this diagram, draw the linear separator with maximal margin (use your intuition about what it should be). Write down the equation for this line.
2. Write down the constrained optimization problem that defines the SVM learning problem (primal form). Do not write the abstract equations that we saw in the lecture, but the concrete equations that we obtain for this particular learning problem. Use w_i for the coefficient of x_i, and b for the bias term.
3. Write down the KKT conditions. Can you solve the optimization problem based on these?
4. The "kernel matrix" is a matrix that contains in cell i, j the dot product of the i'th and j'th instances, $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$. Write down this matrix.
5. Now write down the concrete dual function $\tilde{f}$ that needs to be optimized. Note how you can make use of the kernel matrix that you just calculated. Find the optimum. Derive the primal solution from this (the parameter values w_1, w_2 and b). Does it correspond with your answer to question 1?

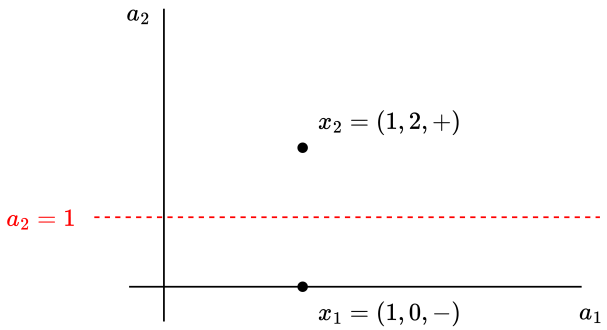


Figure 1: Graph for 4.6.1

Solution 4.6

1. The graph can be found in figure 1.

2. We can define the separator more generally as $w_1x_1 + w_2x_2 + b = 0$. This separator classifies any instance $\mathbf{x}$ such that $w_1x_1 + w_2x_2 + b > 0$ as positive and $w_1x_1 + w_2x_2 + b < 0$ as negative. The maximal margin classifier can be found by minimizing the 2-norm of the weights $||w||_2^2$ under the constraint that
$$y_i(w_1x_{1i} + w_2x_{2i} + b) \geq 1$$
In which we can fill in data instances x_1 and x_2. Thus, we want to minimize $f(w_1, w_2, b) = \frac{1}{2}(w_1^2 + w_2^2)$ with $-w_1 - b \geq 1$ and $w_1 + 2w_2 + b \geq 1$.

3. To find the KKT conditions, we need to rewrite our optimization problem to the *canonical form*, which means rewriting our objective and constraints to the standard form seen in the lecture:
$$\min f(w_1, w_2, b) \text{ such that } g_i(w_1, w_2, b) \geq 0 \text{ and } h_i(w_1, w_2, b) = 0$$
$f = \frac{1}{2}(w_1^2 + w_2^2)$ is our original objective. The constraints of the SVM problem can be adapted to fit the form of g in the KKT conditions: $-w_1 - b - 1 \geq 0$ and $w_1 + 2w_2 + b - 1 \geq 0$. There are no constraints of the type h.

   Calculating the gradients of f, g_1 towards the model parameters, we find
$$\nabla_{(w_1,w_2,b)}f = \begin{bmatrix} w_1 \\ w_2 \\ 0 \end{bmatrix}, \nabla_{(w_1,w_2,b)}g_1 = \begin{bmatrix} -1 \\ 0 \\ -1 \end{bmatrix}, \nabla_{(w_1,w_2,b)}g_2 = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}.$$
   With $L((w_1, w_2, b), \lambda, \nu) = f(w_1, w_2, b) - \sum_i \lambda_i g_i(w_1, w_2, b) - \sum_i \nu_i h_i(w_1, w_2, b)$, we can now write the KKT conditions as follows:
$$\begin{bmatrix} w_1 \\ w_2 \\ 0 \end{bmatrix} - \lambda_1 \begin{bmatrix} -1 \\ 0 \\ -1 \end{bmatrix} - \lambda_2 \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = 0$$
$$-w_1 - b - 1 \geq 0$$
$$w_1 + 2w_2 + b - 1 \geq 0$$
$$\lambda_1 \geq 0,\ \lambda_2 \geq 0$$
$$\lambda_1(-w_1 - b - 1) = 0$$
$$\lambda_2(w_1 + 2w_2 + b - 1) = 0$$
   This system of equations can be solved by inspection, as we can see that $\lambda_1 = \lambda_2$ from the first condition (3rd row), which leads to $w_1 = 0$, $b = -1$, and $w_2 = 1$. Filling in the variables in the separator gives $0x_1 + 1x_2 - 1 = 0$. Or, any instance with $x_2 > 1$ gets classified as positive.

4. Remember how the dot product of vectors $\begin{bmatrix} a \\ b \end{bmatrix}$ and $\begin{bmatrix} c, & d \end{bmatrix}$ is defined as $ac + bd$. Our kernel matrix K becomes:
$$K = \begin{bmatrix} \mathbf{x}_1 \cdot \mathbf{x}_1 & \mathbf{x}_1 \cdot \mathbf{x}_2 \\ \mathbf{x}_2 \cdot \mathbf{x}_1 & \mathbf{x}_2 \cdot \mathbf{x}_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 5 \end{bmatrix}$$

5. We are going to use the *kernel trick* (though our instances are linearly separable, we can still use the kernel trick). Because we have defined K using the dot product, we don't have to show it is a valid kernel using Mercer's theorem. Now we are going to solve the dual optimization problem, substituting $\mathbf{x}_i\mathbf{x}_j$ with $K(\mathbf{x}_i, \mathbf{x}_j)$. The constraint of the dual problem is
$$\sum_i \lambda_i y_i = 0 \iff \lambda_1 \cdot -1 + \lambda_2 \cdot 1 = 0 \iff \lambda_1 = \lambda_2$$
   and the objective to maximize is
$$\max \bar{f}(\lambda) = \max \textstyle\sum_i \lambda_i - \frac{1}{2} \sum_i \sum_j \lambda_i\lambda_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_2) =$$
$$\max \lambda_1 + \lambda_2 - \tfrac{1}{2}(\lambda_1^2 K(\mathbf{x}_1, \mathbf{x}_1) + \lambda_2^2 K(\mathbf{x}_2, \mathbf{x}_2) - \lambda_1\lambda_2 K(\mathbf{x}_1, \mathbf{x}_2) - \lambda_2\lambda_1 K(\mathbf{x}_2, \mathbf{x}_1))$$

By filling in the values from the kernel matrix and substituting the constraint $\lambda_1 = \lambda_2$, we obtain

$$\max 2\lambda_1 - \frac{1}{2}(\lambda_1^2 + 5\lambda_1^2 - 2\lambda_1^2)) = 2\lambda_1 - 2\lambda_1^2$$

To optimize this for $\lambda_1 \geq 0$, we calculate the derivative and equate it to zero:

$$\frac{d}{d\lambda_1}(2\lambda_1 - 2\lambda_1^2) = 0 \iff 2 - 4\lambda_1 = 0 \iff \lambda_1 = \frac{1}{2} \ (= \lambda_2)$$

You can quickly verify that this value gives a maximum by using a plotting tool, or checking surrounding values $\lambda = 0$ and $\lambda = 2$, which both lead to a smaller objective. $\lambda_1 = \frac{1}{2}$ therefore gives a maximum.
Knowing λ_1 and λ_2, we can calculate w_1, w_2:

$$\mathbf{w} = \sum_i \lambda_i y_i \mathbf{x} = \frac{1}{2} \cdot -1 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \frac{1}{2} \cdot 1 \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

and b:

$$b = -1 - \mathbf{w} \cdot \mathbf{x_1} = 1 - \mathbf{w} \cdot \mathbf{x_2} = -1$$

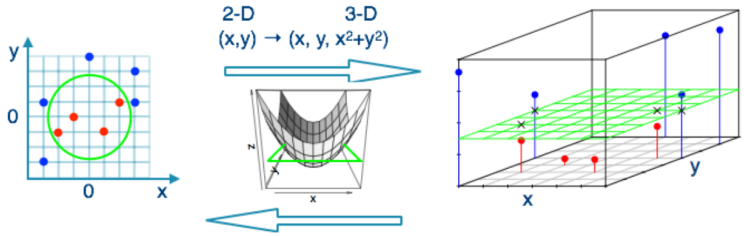
If we fill this into our original equality for the separator (question 2), $w_1 x_1 + w_2 x_2 + b = 0$, we obtain $x_2 - 1 = 0$, which is equivalent to our separator from question 3.

◇

4.7 SVM Kernel Trick: Mapping to Higher Dimensions

To learn a non-linear SVM, the input data is transformed to a higher-dimensional space and a linear SVM is learned in this new space. The linear separator in the higher-dimensional space then corresponds to a non-linear separator in the original space. The hope is that the data is actually linearly separable after the transformation.

The following visual example was given in the slides.



We also saw that in the dual formulation, the transformation only appeared in dot-product pairs: $\Phi(x) \cdot \Phi(x')$ for two input instances x and x'. As a result, we saw that it is not necessary to explicitly specify Φ, using just a kernel function $K(x, x')$ is sufficient. One requirement for such a kernel is that it corresponds to a dot product in a transformed space.

Consider the following kernel K:

$$K(v_1, v_2) = (v_1 \cdot v_2 + 1)^2,$$

where v_1 and v_2 are two vectors in $\mathbb{R}^2$. Show that this kernel corresponds to

$$\langle \Phi(v_1), \Phi(v_2)) \rangle_F$$

where Φ is the following transformation:

$$\Phi : \mathbb{R}^2 \to \mathbb{R}^{3\times 3} : v = (x, y) \mapsto \Phi(v) = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \begin{bmatrix} x & y & 1 \end{bmatrix},$$

and $\langle A, B\rangle_F$ is the Frobenius inner product defined on two matrices A, and B:

$$\langle A, B\rangle_F = \text{trace}(A^T B) = \sum_i \sum_j A_{ji} B_{ji}.$$

Note that Φ maps a vector v, after extending it with the constant 1, to its outer product.

Solution 4.7 Applying Φ to a vector $v = (x, y)$ yields the matrix

$$\Phi(v) = \begin{bmatrix} x^2 & xy & x \\ xy & y^2 & y \\ x & y & 1 \end{bmatrix}$$

which is symmetric.

The Frobenius inner product of two matrices A and B can be rewritten into

$$A \cdot B = \text{tr}(A^{\mathrm{T}} B) = \sum_i (A^{\mathrm{T}} B)_{ii} = \sum_i \sum_j A^{\mathrm{T}}_{ij} B_{ji} = \sum_i \sum_j A_{ji} B_{ji}$$

and thus amounts to taking the usual componentwise vector product as if the two matrices were vectors.

We can then expand $K(v_1, v_2)$:

$$\begin{aligned} K(v_1, v_2) &= (x_1 x_2 + y_1 y_2 + 1)^2 \\ &= (x_1 x_2)^2 + 2 x_1 x_2 (y_1 y_2 + 1) + (y_1 y_2 + 1)^2 \\ &= x_1^2 x_2^2 + 2 x_1 x_2 y_1 y_2 + 2 x_1 x_2 + y_1^2 y_2^2 + 2 y_1 y_2 + 1, \end{aligned}$$

and $\langle \Phi(v_1), \Phi(v_2)\rangle_F$:

$$\begin{aligned} \langle \Phi(v_1), \Phi(v_2)\rangle_F &= \left\langle \begin{bmatrix} x_1^2 & x_1 y_1 & x_1 \\ x_1 y_1 & y_1^2 & y_1 \\ x_1 & y_1 & 1 \end{bmatrix}, \begin{bmatrix} x_2^2 & x_2 y_2 & x_2 \\ x_2 y_2 & y_2^2 & y_2 \\ x_2 & y_2 & 1 \end{bmatrix} \right\rangle \\ &= x_1^2 x_2^2 + x_1 y_1 x_2 y_2 + x_1 x_2 + x_1 y_1 x_2 y_2 + y_1^2 y_2^2 + y_1 y_2 + x_1 x_2 + y_1 y_2 + 1 \\ &= x_1^2 x_2^2 + 2 x_1 x_2 y_1 y_2 + 2 x_1 x_2 + y_1^2 y_2^2 + 2 y_1 y_2 + 1 \end{aligned}$$

which is equal to $K(v_1, v_2)$. ◇